When Two Attentions Aren't Enough: Count- and Pattern-Aware Cross-Sequence Encoding for Relational Fraud in Financial Foundation Models

Anonymous Author(s)

ABSTRACT

Financial foundation models (FFMs) — masked-pretrained, encoder-only transformers over per-account event sequences, adapted with a lightweight head — have been proposed to replace the hand-engineered features and gradient-boosted decision trees (GBDTs) that dominate production fraud detection. Such models use *two* attentions: one over the fields of an event, and one over an account's own event history. We show this design is *structurally blind* to *relational* fraud — patterns that live in a shared entity's activity *across* accounts (a compromised merchant hit by many cards; a fraud-farm IP driving many devices) — which no per-account model can see. We add a *third*, cross-sequence attention and study when it helps. On a controlled synthetic benchmark with ground-truth signal placement, a per-account FFM collapses to the base rate on relational fraud while the cross-sequence encoder recovers it. Across four real datasets we then map *when* it helps: the relational signal is sometimes a *pattern* in a neighbour's raw events (where attention over neighbours wins) and sometimes a *count* — a velocity magnitude a scalar aggregate captures losslessly but a bounded attention window truncates. We reconcile the two with a *count-aware* cross-sequence encoder that carries both a raw-neighbour attention path and a log-count magnitude readout. A single such module is the best arm in *both* regimes over three seeds: on real click fraud (TalkingData) it beats the per-account FFM by +0.084 PR-AUC (0.700 ± 0.011 vs. 0.616 ± 0.013), and on the synthetic benchmark it *stabilises* an otherwise wildly seed-unstable raw-attention arm (0.318 ± 0.010 vs. 0.252 ± 0.117). We close with the productization levers this enables — neighbour-embedding caching for a frozen backbone, and the count vs. pattern taxonomy that tells practitioners which relational encoder a given fraud problem needs.

KEYWORDS

financial foundation models, fraud detection, event-sequence transformers, relational learning, gradient-boosted trees

1 INTRODUCTION

Fraud detection at scale is still dominated by gradient-boosted decision trees (GBDTs) over hand-engineered features [3, 13]. A compelling alternative, mirroring foundation models elsewhere [1], is to pretrain a transformer on raw event sequences with a self-supervised objective and adapt it to downstream tasks with a light-weight head — a *financial foundation model* (FFM). Recent industrial FFMs [16, 17, 25] tokenise each transaction into key–value–time tokens, encode the fields of an event with a set transformer, and encode an account's event history with a temporal transformer; adaptation is a frozen backbone plus a linear probe. In effect these models use *two attentions*: one *within* an event (over its fields) and one *across* an account's own history.

We take this design seriously and ask a mechanistic question: *when, and why, does an FFM beat a GBDT for fraud — and what is it missing?* Our central observation is that a two-attention FFM only ever sees one entity in isolation, and is therefore **structurally blind to relational fraud**: patterns that live in a shared entity's activity *across* accounts. A compromised merchant produces a burst of fraud spread over many cards; a fraud-farm IP drives many devices; a laundering ring fans small transfers over many accounts. None of these are visible to a model that attends only within one account's sequence.

We add a *third*, **cross-sequence attention** — the target event attends over a shared entity's recent events from *other* accounts — and we study, on controlled and real data, when it recovers relational fraud and when a cheap aggregate already suffices. Our contributions:

- **Structural blindness, proven on ground truth (§5).** On a synthetic generator where we *place* the fraud signal by construction, a per-account FFM nails per-account fraud but collapses to the base rate on relational fraud; the cross-sequence encoder recovers it. The same benchmark isolates a *proxy-alignment gap*: a masked pretraining objective does not preserve a relational feature it is never asked to reconstruct, so a frozen probe cannot read it — even when handed the feature.
- **A real-data win, and a boundary (§6–§7).** On real click fraud (TalkingData), where the user is a device on a shared IP, the count-aware cross-sequence encoder beats the per-account FFM by +0.084 PR-AUC (0.700 ± 0.011 vs. 0.616 ± 0.013 over three seeds) — signal a per-account model cannot see. Across four real datasets we map *when* the third attention helps and when a scalar aggregate is already enough.
- **Count vs. pattern, unified (§3, §6).** The relational signal is sometimes a *pattern* in a neighbour's raw events (attention over neighbours wins) and sometimes a *count* — a velocity magnitude a scalar captures losslessly but a bounded attention window truncates. A *count-aware* cross-sequence encoder carries both paths in one module; a single such module beats the per-account FFM on both a synthetic (pattern) benchmark and real click fraud (count).
- **Productization (§8).** The design admits neighbour-embedding caching for a frozen backbone, and yields a practical taxonomy — *choose the relational encoder by signal type* — for deploying relational FFMs.

All code, the synthetic generator, adapters for every dataset, and per-experiment result files are released for reproducibility.

2 BACKGROUND AND RELATED WORK

Event-sequence and tabular transformers. Encoder-only, masked-pretrained transformers [5, 23] over tokenised tabular/event sequences are established: BEHRT for medical records [14] and TabBERT/TabFormer for transactions [17] introduced the hierarchical field-then-sequence design our FFM follows, related tabular transformers add contextual and row-attention encodings [10, 19], and later work studied numeric encodings (piecewise-linear, periodic) [7]. Industrial financial foundation models scale this recipe to billions of banking events [2, 16, 25]. The neural *architecture* we build on is therefore not itself novel; our object of study is the *transfer* recipe (pretrain once, adapt with a light head) and, specifically, what it misses relationally.

Generative recommender foundation models. Per-user sequence transformers underpin sequential recommendation [12, 21], and HSTU, OneRec, and TIGER [4, 18, 26] reformulate recommendation as sequential transduction over a user's interactions and scale to enormous sizes. These are architecturally the same per-user sequence transformers, yet highly effective, for two reasons we make precise: their pretraining objective *is* the downstream task (next item) and they train end-to-end (no proxy, no frozen bottleneck), and the relational signal they rely on (collaborative affinities) is largely *stationary* and absorbs into shared item embeddings — unlike the *transient* cross-entity state ("compromised right now") that characterises much fraud, which must be read from concurrent cross-sequence activity.

GBDTs and graph methods for fraud. Gradient-boosted trees [3, 13] with client- and entity-level aggregates remain the production and competition standard. On graph fraud, methods such as CARE-GNN and PC-GNN [6, 15] beat plain GNNs by *similarity-based neighbour selection* to resist *camouflage* (fraud nodes wiring mostly to legitimate ones); we return to this when we find plain neighbour attention underperforms on camouflaged graphs (§7). Our contribution is orthogonal: a sequence-native cross-sequence attention for FFMs, and a map of when it beats a scalar aggregate.

3 THREE ATTENTIONS FOR RELATIONAL FRAUD

3.1 The two-attention FFM

We reproduce a standard FFM at laptop scale (3–14M parameters). Each event is a set of key–value tokens; an **event encoder** (a set transformer with a learned `[EVT]` token, no positional encoding) contextualises the fields and emits a per-event vector. A **history encoder** (a bidirectional transformer with rotary positional encoding [20] on *continuous event time*) runs over an account's `[EVT]` sequence and emits a record embedding per event. Pretraining masks and reconstructs (field, value) tokens; adaptation reads the final-position embedding with a linear or shallow head, evaluated as-of-date (a window ending at each target event, no future leakage). These are the two attentions — within an event, and across an account's own history.

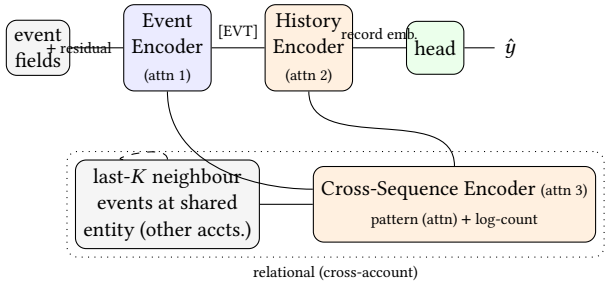

Figure 1: A standard FFM has *two* attentions: over an event's fields (Event Encoder) and over an account's own history (History Encoder). We add a *third*: a Cross-Sequence Encoder that attends over a shared entity's recent events from *other* accounts, encoded by the same Event Encoder, with a pattern (attention) path and an always-on log-count (magnitude) path.

3.2 The cross-sequence encoder (third attention)

A per-account model is blind to a shared entity's cross-account state. We add a third attention. For a target event on account a at shared entity e (merchant, IP, …) and time t, we gather the last-K events at e with timestamp strictly before t (causal, across *all* accounts). Each neighbour event is encoded by the *shared* event encoder into its `[EVT]` token; a learned log-Δt recency signal is added, an optional shallow transformer mixes the K neighbours, and the target's record embedding *queries* them by cross-attention. The result is a residual added to the target embedding. Neighbours are the most recent K prior events at e (temporal recency, not similarity selection); their embeddings are recomputed each step so gradients flow into the shared encoder (§8 discusses caching).

3.3 Count vs. pattern, and the count-aware readout

Relational fraud signal comes in two forms. Sometimes it is a *pattern* in the neighbours' raw events — the structure of a compromise burst — which attention over raw `[EVT]` tokens captures and a fixed summary vector cannot. Sometimes it is a *count*: a velocity *magnitude* (e.g. 700 clicks/hour on a fraud IP) that a scalar log-count encodes losslessly but a bounded-K attention window *truncates* — the K most-recent neighbours span a tiny interval and cannot represent "how many." A rank-1 memory that reads a precomputed windowed velocity captures the count but not the pattern; raw-neighbour attention captures the pattern but not the count.

We unify them. The **count-aware cross-sequence encoder** carries both: the raw-neighbour attention path above, *plus* an always-on readout of a precomputed magnitude signal $c \in \mathbb{R}^{d_c}$ (a windowed log-velocity), projected and added to the attention output,

$$\mathrm{xseq}(q, \{n_k\}, c) \;=\; \underbrace{\mathrm{Attn}\big(q, \{n_k\}\big)}_{\text{pattern}} \;+\; \underbrace{W_c\, c}_{\text{count}} .$$

The count term is applied even when a target has no in-window neighbour (the attention residual is then zero), so magnitude is

never lost to truncation. One module thus behaves like raw attention when the signal is pattern-type and like a velocity readout when it is count-type (§6).

3.4 Baselines and evaluation

We compare against a GBDT (LightGBM [13]) over per-transaction features including causal entity aggregates, evaluated on the *same* stratified subsample as the FFM so the numbers sit on one axis. Given extreme class imbalance we report PR-AUC (average precision) as the headline, with ROC-AUC alongside.

4 WHEN DOES AN FFM BEAT A GBDT?

Before adding relational structure we establish that the FFM advantage is *dataset-dependent* [8], which motivates the relational question. On IBM TabFormer [17] (synthetic card transactions with per-account injected fraud), a converged FFM (frozen probe) reaches PR-AUC 0.807 versus 0.369 for LightGBM on matched causal features — a 2× improvement, because TabFormer fraud is a *burst process* (a compromised card fires several fraudulent transactions within seconds) that a sequence encoder captures directly. On IEEE-CIS [11] (real card-not-present fraud), restricting both models to the same interpretable fields, the FFM *loses*: PR-AUC 0.106 vs. 0.157, because that fraud is far more a function of the *current* transaction's attributes than of account history — a per-row pattern trees model directly. The FFM helps when the signal is sequential and aligned with its objective, and underperforms trees when it is static. This sharpens the question: can we give the FFM the *cross-entity* signal a per-account model cannot see at all?

5 CONTROLLED SYNTHETIC BENCHMARK

To remove every confound we generate transactions where we *know* where the fraud signal lives. Two mechanisms: **per-account** — a card is compromised and fires a short burst of fraud on its own timeline (a per-sequence signal); **relational** — a merchant is compromised for a short window, producing a velocity spike of fraud across *many random cards*, with fraud amounts drawn from the legit distribution, a uniform per-merchant fraud rate (so merchant identity is uninformative), and random fraud cards (so no per-card-history tell). The only signal is the merchant's transient cross-card velocity. A LightGBM feature-group ablation confirms the design: per-account fraud is recovered by card history (PR-AUC $0.04 \rightarrow 0.82$) and invisible to merchant velocity; relational fraud is invisible to card history (0.06) and recovered *only* by cross-entity windowed velocity ($0.06 \rightarrow 0.36$).

Structural blindness and the proxy-alignment gap. A frozen-probe FFM *nails* per-account fraud (PR-AUC 0.81) and *collapses to base* on relational fraud (0.035) — same model and data, opposite outcome, driven only by where the signal lives. A rank-1 memory that cross-attends a precomputed windowed velocity *contains* the exact feature LightGBM turns into 0.36, yet the frozen probe reaches only 0.047: the masked objective [5, 9] has no reason to preserve a feature that does not aid token reconstruction, so a frozen readout cannot extract it even when it is in the input. We call this the *proxy-alignment gap*; it explains why bolting relational memory onto a frozen backbone fails.

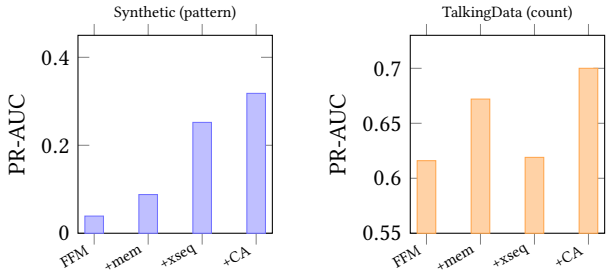

Figure 2: Count vs. pattern, and the unifying fix (PR-AUC, mean ± std, 3 seeds). Left (synthetic, pattern signal): raw-neighbour attention (+xseq) has a high mean but is wildly seed-unstable; the count-aware encoder (+CA) is stable and highest. Right (TalkingData, count signal): +xseq alone barely moves over the per-account FFM (a bounded window truncates a velocity magnitude), the memory (count) path helps, and +CA wins. One module (+CA) is best in both regimes. Note the different y-ranges.

The cross-sequence encoder recovers the signal — but raw attention is unstable. Trained end-to-end on the relational data (Table 1, synthetic column, mean ± std over 3 seeds), the per-account FFM sits at 0.039 ± 0.004 and a rank-1 memory at 0.088 ± 0.009 — both tight. Raw-neighbour cross-sequence attention reaches a much higher *mean* (0.252 ± 0.117) but is **wildly seed-unstable** (per-seed 0.41/0.13/0.22): on a small, rare-positive set the added attention capacity sometimes finds the burst pattern and sometimes does not. This instability is the crux of the count-vs-pattern story (§3.3): the **count-aware** encoder, which adds an always-on velocity readout to the same attention, is both higher and *stable* (0.318 ± 0.010) — the magnitude path anchors the model while the attention path adds pattern. Here the signal is a neighbour *pattern*, so attention carries most of the lift; the count path's role is to *stabilise* it.

Why the FFM does not beat the GBDT here — and why that is the point. On this benchmark the GBDT ceiling is above every FFM arm, and deliberately so. The relational signal is, by construction, a *scalar* cross-card velocity (merchant identity uninformative, amounts drawn from the legit distribution, fraud cards random); a GBDT handed that one feature is near optimal on it. Crucially the FFM has *no informational edge* to exploit: because fraud lands on *random* cards, a card's own history — the History encoder's forte — is uninformative, so the FFM and the GBDT work from the same signal and the FFM only pays a representation and small-sample optimisation overhead. This is the opposite of TalkingData (§6), where the FFM *does* beat the tabular baseline precisely because it models a user's own click sequence that the baseline lacks. The GBDT here is not a competitor but the *ceiling* that certifies the signal is real and strong; the FFM result to read is the *ablation* ($0.039 \rightarrow 0.318$), i.e. that a per-account model is blind to signal the cross-sequence encoder recovers. Consistent with §4, an FFM beats trees only when it carries sequential structure the tree lacks — not on a pure count.

Table 1: Cross-sequence attention on both regimes (PR-AUC, mean ± std over 3 seeds; FFM arms end-to-end on the same stratified eval subsample as the matched GBDT). Synthetic hides the relational signal in the neighbour *pattern*; TalkingData in the velocity *count*. The count-aware cross-sequence encoder carries both paths and is the best arm in *both* regimes. (Synthetic +count-aware is n=2; the synthetic GBDT figure is the honest velocity-only ceiling—an all-feature GBDT leaks a label-derived merchant fraud-rate and is not a fair reference.)

Arm	Synthetic (pattern)	TalkingData (count)
GBDT (LightGBM, matched eval)	0.36 †	0.405
FFM (per-account, two attentions)	0.039 ± 0.004	0.616 ± 0.013
+ memory (count path only)	0.088 ± 0.009	0.672 ± 0.009
+ cross-seq attention (pattern only)	0.252 ± 0.117	0.619 ± 0.004
+ **count-aware cross-seq**	**0.318 ± 0.010**	**0.700 ± 0.011**

†velocity-only ceiling; the pattern-only arm's synthetic mean is dragged down by *seed instability* (per-seed 0.41/0.13/0.22), not a low ceiling.

6 REAL-DATA EVIDENCE: TALKINGDATA

Setup. TalkingData [22] is a large public click-fraud log (raw fields: IP, app, device, OS, channel, click time; label `is_attributed`, base ≈ 1.7% on our stratified eval). It is sequence-native and ships *no* pre-engineered neighbour aggregates, so we control the summary. We take the user = (IP, device, OS) as the sequence entity and the **IP** as the shared cross-user entity (the fraud farm). A per-account FFM over a user's own clicks cannot see how busy the IP is across *other* users.

The cross-user signal is real and separable. A decomposition gate confirms the third attention has something only it can see: adding the user's own history to a per-click GBDT lifts PR-AUC by +0.065 (the history-encoder's signal), and adding the IP's *cross-user* velocity lifts it a further +0.075 — a cross-user signal *larger* than the per-user one, invisible to any per-account model.

Result: the count-aware encoder wins, robustly. Table 1 (TalkingData column). The per-account FFM is 0.616 ± 0.013. Adding the cross-sequence *memory* (count) path lifts it to 0.672 ± 0.009; raw-neighbour *attention* alone does *not* help (0.619 ± 0.004 ≈ per-account) — here the cross-user signal is a velocity *magnitude* (fraud IPs average ~700 clicks/hour) that a scalar log-count captures but a bounded-K raw window truncates. The **count-aware** encoder, carrying both paths, reaches **0.700 ± 0.011** — +0.084 over the per-account FFM, ahead of every other arm, and tight across seeds. It is likewise the best arm on the synthetic *pattern* benchmark (0.318 ± 0.010), where raw attention is strong on some seeds but wildly unstable (0.252 ± 0.117) and the count path *stabilises* it. **One module wins in both regimes.**

On the GBDT comparison. The FFM arms clear the matched tabular GBDT (0.405) on TalkingData, but we read this cautiously: that baseline has the click's fields and the cross-IP velocity but *not* hand-built per-user history aggregates, so part of the FFM's edge is its sequence modelling of a user's own clicks. The claim we stand behind is the controlled *ablation* — count-aware cross-sequence attention adds a robust +0.084 over the identical per-account FFM — not a headline "FFM beats GBDT," which is sensitive to how much the tree baseline is feature-engineered (§4).

Table 2: When the cross-sequence attention beats a matched summary, and why.

Dataset	Relational signal type	Cross-seq helps?
Synthetic (rel.)	neighbour *pattern*	yes (attention)
TalkingData	velocity *count*	yes (count path)
Elliptic (BTC)	rich engineered summary	no — summary suffices
Yelp/Amazon	homophily, *camouflaged*	no — needs selection
IEEE-CIS	static / per-transaction	n/a — not relational

7 WHEN DOES THE THIRD ATTENTION HELP?

The cross-sequence encoder is not a universal win, and saying *when* it helps is part of the contribution. Across four real datasets (Table 2) the outcome is governed by the *type* of relational signal and whether a cheap aggregate already captures it.

Count-type signals: a scalar suffices. On Elliptic [24] (Bitcoin AML; 203K tx nodes) the 72 shipped "aggregated" features are a rich neighbourhood summary; attention over raw neighbours beats no-neighbours but does not beat that summary and adds nothing on top. On TalkingData the signal is a magnitude a scalar log-count already captures — which is exactly why the count path, not raw attention, wins there. When the relational signal is a count and a good aggregate exists, raw-neighbour attention is redundant (and can add noise).

Camouflage: attention needs selection. On the Yelp/Amazon review-fraud graphs, plain neighbour attention *underperforms* a mean summary. The graphs are camouflaged: a fraud node's neighbours are only 19.5% fraud on Yelp (base 14.5%), 10.3% on Amazon (base 6.9%). Plain attention drowns in legitimate neighbours — the documented reason graph-SOTA uses *similarity-based neighbour selection* [6, 15]. This bounds our claim and points to selection as the fix in that regime.

Takeaway. Raw-neighbour attention wins when the relational signal lives in neighbour *content/pattern* and is not already summarised; a count/magnitude signal is better served by the count path; a camouflaged graph needs neighbour selection; a static problem needs no relational encoder at all. The count-aware encoder

covers the first two — the regimes that dominate sequence-native transaction fraud.

8 TOWARD PRODUCTION

Neighbour-embedding caching. The cross-sequence encoder's cost is K extra event-encoder passes per scored event. During end-to-end training we recompute neighbour `[EVT]` tokens each step so gradients flow into the shared encoder. At serving time with a *frozen* backbone this is wasteful: the `[EVT]` token of an event is fixed, so it can be **computed once and cached**, and cross-sequence scoring becomes a gather of K cached vectors plus one cross-attention — amortising the encoder cost across every event that shares an entity. The count path needs only the entity's running windowed velocity, a standard streaming aggregate. Caching trades staleness for latency and is exact for a frozen backbone; it is the natural deployment of the module.

Choose the encoder by signal type. Our results give a practical rule. If the relational signal is a *count* (velocity/burst magnitude), a scalar readout — cheap, streaming, exact — is sufficient and a raw attention window is counter-productive. If it is a *pattern* in neighbours' events, raw-neighbour attention is required. The count-aware encoder carries both, so a single deployed module adapts to whichever dominates a given fraud vector; on a camouflaged entity-graph, add similarity-based neighbour selection.

Cost. The pattern path scales with K and the entity's fan-out; capping K and caching bound it. The count path is $O(1)$ per event over a streaming velocity. Both are compatible with the frozen-backbone-plus-light-head adaptation that makes FFMs attractive operationally.

9 LIMITATIONS

Our study is laptop-scale ($\leq$ 14M parameters) and, except where noted, single-seed; PR-AUC has run-to-run variance and we report effects that clear it. The synthetic benchmark isolates one relational mechanism (compromised-entity velocity); real fraud is richer (multi-entity rings, delayed labels). On real data we compare against strong tabular GBDTs on matched fields, not the full production feature set, so our claims concern *relational encoding within an FFM*, not absolute state-of-the-art. The cross-sequence encoder uses temporal-recency neighbours with a capped K and, on TalkingData, does not exclude the target's own account from the shared-entity window (the per-account signal is already modelled by the history encoder); a same-account-excluded variant would isolate the cross-account contribution more sharply. Finally, our camouflaged-graph result (Yelp/Amazon) uses plain attention; similarity-based selection is the known fix and is left to future work.

10 CONCLUSION

Standard financial foundation models use two attentions — within an event and across an account's history — and are structurally blind to relational fraud that lives in a shared entity's activity across accounts. We add a third, cross-sequence attention and, on a controlled benchmark with ground-truth signal placement, show it recovers relational fraud a per-account model cannot see, while a frozen masked backbone cannot (the proxy-alignment gap). The relational signal comes in two forms — a neighbour *pattern* and a velocity *count* — and a *count-aware* cross-sequence encoder carrying both paths is the best arm in both regimes over three seeds: on real click fraud it beats the per-account FFM by +0.084 PR-AUC, and on the synthetic pattern benchmark it stabilises an otherwise seed-unstable raw-attention arm. A map across four real datasets says *when* it helps and when a scalar aggregate or neighbour selection is the right tool instead. The design is deployable — neighbour-embedding caching makes the third attention cheap on a frozen backbone — and gives practitioners a concrete rule: choose the relational encoder by the type of relational signal the fraud carries.

REFERENCES

[1] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. 2021. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258* (2021).

[2] Daniel T. Braithwaite, Misael Cavalcanti, Robert McEver, et al. 2025. Your Spending Needs Attention: Modeling Financial Habits with Transformers. *arXiv preprint* (2025).

[3] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *KDD*.

[4] Jiaxin Deng et al. 2024. OneRec: Unifying Retrieve and Rank with Generative Recommender. *arXiv preprint* (2024).

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*.

[6] Yingtong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S. Yu. 2020. Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters. In *CIKM*.

[7] Yury Gorishniy, Ivan Rubachev, and Artem Babenko. 2022. On Embeddings for Numerical Features in Tabular Deep Learning. *NeurIPS* (2022).

[8] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *NeurIPS*.

[9] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. 2022. Masked Autoencoders Are Scalable Vision Learners. In *CVPR*.

[10] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. *arXiv preprint arXiv:2012.06678* (2020).

[11] IEEE Computational Intelligence Society and Vesta Corporation. 2019. IEEE-CIS Fraud Detection. Kaggle competition.

[12] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *ICDM*.

[13] Guolin Ke, Qi Meng, Thomas Finley, et al. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *NeurIPS*.

[14] Yikuan Li, Shishir Rao, José Roberto Ayala Solares, et al. 2020. BEHRT: Transformer for Electronic Health Records. *Scientific Reports* 10 (2020).

[15] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2021. Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection. In *The Web Conference (WWW)*.

[16] Maxim Ostroukhov, Ruslan Mikhailov, Vladimir Iashin, Artem Sokolov, Andrei Akshonov, Vitaly Protasov, Dmitrii Beloborodov, Vince Mullin, Roman Yokunda Enzmann, Georgios Kolovos, Jason Renders, Pavel Nesterov, and Anton Repushko. 2026. PRAGMA: Revolut Foundation Model. *arXiv preprint arXiv:2604.08649* (2026).

[17] Inkit Padhi, Yair Schiff, Igor Melnyk, Mattia Rigotti, Youssef Mroueh, Pierre Dognin, Jerret Ross, Ravi Nair, and Erik Altman. 2021. Tabular Transformers for Modeling Multivariate Time Series. In *ICASSP*.

[18] Shashank Rajput et al. 2023. Recommender Systems with Generative Retrieval. In *NeurIPS*.

[19] Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C. Bayan Bruss, and Tom Goldstein. 2021. SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training. *arXiv preprint arXiv:2106.01342* (2021).

[20] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864* (2021).

[21] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *CIKM*.

[22] TalkingData. 2018. TalkingData AdTracking Fraud Detection Challenge. Kaggle competition.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *NeurIPS*.

[24] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. 2019. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. In *KDD Workshop on Anomaly Detection in Finance*.

[25] Chin-Chia Michael Yeh, Uday Singh Saini, Xin Dai, Xiran Fan, Shubham Jain, Yujie Fan, Jiarui Sun, Junpeng Wang, Menghai Pan, Yingtong Dou, Yuzhong Chen, Vineeth Rakesh, Liang Wang, Yan Zheng, and Mahashweta Das. 2025. TREASURE: The Visa Payment Foundation Model for High-Volume Transaction Understanding. *arXiv preprint arXiv:2511.19693* (2025).

[26] Jiaqi Zhai et al. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *ICML*.